

Grafana Faro

React Application Integration Guide

Real User Monitoring (RUM) Setup

1. Overview

This document provides step-by-step instructions for integrating Grafana Faro SDK into a React application. Once integrated, the application will automatically send Real User Monitoring (RUM) data — including page load times, user interactions, API call traces, JavaScript errors, and session data — to the central observability platform.

What you will get after integration:

- Page load time and Core Web Vitals (LCP, INP, CLS)
- JavaScript errors — captured automatically
- API call traces — visible in Grafana Tempo
- Application logs — visible in Grafana Loki
- User session tracking — full user journey
- Custom events — button clicks, form submissions, etc.

2. Prerequisites

2.1 Check Node.js Version

Run the following command to verify your Node.js version:

```
node --version
```

Note: Vite-based projects require Node.js 20.19 or higher. CRA-based projects require Node.js 16 or higher. If your version is lower, upgrade before proceeding.

2.2 Identify Your React Setup

Run this command inside your project directory to determine which build tool your project uses:

```
cat package.json | grep -E '"vite"|"react-scripts"'
```

Setup	Output contains	Action
Vite	"vite": "..."	Use VITE_ prefix for env variables
CRA	"react-scripts": "..."	Use REACT_APP_ prefix for env variables

3. SDK Installation

Navigate to your project root and install the Faro packages:

```
npm install @grafana/faro-web-sdk @grafana/faro-web-tracing
```

Verify the packages were installed:

```
cat package.json | grep faro
```

You should see both packages listed under dependencies.

4. Create faro.js

Create a new file named faro.js inside the src/ directory of your project. This file initializes the Faro SDK and must be loaded before the React application renders.

4.1 File Location

```
your-project/  
  src/  
    faro.js      ← create this file  
    main.jsx     (or index.js)  
    App.jsx
```

4.2 faro.js Content

Copy the following code exactly into src/faro.js:

For Vite-based projects

```
import {  
  initializeFaro,  
  getWebInstrumentations,  
  LogLevel,  
} from '@grafana/faro-web-sdk'  
import { TracingInstrumentation } from '@grafana/faro-web-tracing'  
  
let faro = null  
  
export function initFaro() {  
  if (faro) return faro  
  
  faro = initializeFaro({  
    url: import.meta.env.VITE_FARO_URL,  
  
    app: {  
      name:      import.meta.env.VITE_APP_NAME,  
      version:   import.meta.env.VITE_APP_VERSION || '1.0.0',  
      environment: import.meta.env.VITE_ENV || 'production',  
    },  
  
    sessionTrackingConfig: {  
      enabled:    true,  

```

```

    persistent: true,
  },

  instrumentations: [
    ...getWebInstrumentations({
      captureConsole: true,
      captureConsoleDisabledLevels: [LogLevel.DEBUG, LogLevel.TRACE],
      capturePerformance: true,
    }),
    new TracingInstrumentation(),
  ],
})

return faro
}

export { faro }

```

For CRA-based projects

The only difference is how environment variables are read. Replace `import.meta.env` with `process.env`:

```

import {
  initializeFaro,
  getWebInstrumentations,
  LogLevel,
} from '@grafana/faro-web-sdk'
import { TracingInstrumentation } from '@grafana/faro-web-tracing'

let faro = null

export function initFaro() {
  if (faro) return faro

  faro = initializeFaro({
    url: process.env.REACT_APP_FARO_URL,

    app: {
      name: process.env.REACT_APP_NAME,
      version: process.env.REACT_APP_VERSION || '1.0.0',
      environment: process.env.REACT_APP_ENV || 'production',
    },

    sessionTrackingConfig: {
      enabled: true,
      persistent: true,
    },

    instrumentations: [
      ...getWebInstrumentations({
        captureConsole: true,
        captureConsoleDisabledLevels: [LogLevel.DEBUG, LogLevel.TRACE],
        capturePerformance: true,
      }),
      new TracingInstrumentation(),
    ],
  })

  return faro
}

```

```
export { faro }
```

5. Inject Faro into the App

Faro must be initialized before React renders. Open your entry file and add the `initFaro()` call at the very top, before any other imports or code.

5.1 Vite — `src/main.jsx`

```
import { initFaro } from './faro.js' // must be first
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import App from './App.jsx'

initFaro() // initialize before React renders

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

5.2 CRA — `src/index.js`

```
import { initFaro } from './faro.js' // must be first
import React from 'react'
import ReactDOM from 'react-dom/client'
import App from './App'

initFaro() // initialize before React renders

const root = ReactDOM.createRoot(document.getElementById('root'))
root.render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
)
```

Note: The import of `faro.js` must always be the very first import in the file. If React or other libraries load before Faro, some auto-instrumentation may not work correctly.

6. Environment Variables

Create environment variable files in your project root. These files tell the SDK where to send data and how to label your application.

6.1 Vite — .env Files

```
# .env.production
VITE_FARO_URL=https://<faro-collector-url>/collect
VITE_APP_NAME=your-app-name
VITE_APP_VERSION=1.0.0
VITE_ENV=production
```

```
# .env.development (for local testing)
VITE_FARO_URL=http://localhost:8081/collect
VITE_APP_NAME=your-app-name
VITE_APP_VERSION=1.0.0
VITE_ENV=development
```

6.2 CRA — .env Files

```
# .env.production
REACT_APP_FARO_URL=https://<faro-collector-url>/collect
REACT_APP_NAME=your-app-name
REACT_APP_VERSION=1.0.0
REACT_APP_ENV=production
```

Variable	Description
FARO_URL	Provided by your platform team — the Faro collector endpoint
APP_NAME	Your application name (e.g. hcmng-frontend). Use lowercase with hyphens
APP_VERSION	Your application version (e.g. 2.1.0)
ENV	Environment name: development / staging / production

7. Verification

7.1 Browser Network Check

Open your browser developer tools, go to the Network tab, and filter by the word collect. When you load the application or click anything, you should see POST requests going to the Faro collector URL with HTTP status 200.

What to look for in Network tab:

URL: https://<faro-collector-url>/collect

Method: POST

Status: 200 OK

If you see a CORS error, contact your platform team to whitelist your domain.

7.2 Grafana — Check Logs (Loki)

Navigate to your Grafana instance. Go to Explore, select the Loki datasource, and run the following query:

```
{service_name="your-app-name"}
```

You should see log entries appearing within 30 seconds of the application loading.

7.3 Grafana — Check Traces (Tempo)

In Grafana Explore, select the Tempo datasource, choose Search query type, and set Service Name to your-app-name. Traces should appear for every page load and API call made from the browser.

8. Troubleshooting

Problem	Cause	Solution
No data in Grafana	Check browser Network tab for POST to /collect	Verify VITE_FARO_URL is set correctly in .env
CORS error in browser	Browser blocked the request to collector	Contact platform team — your domain needs to be whitelisted
Status 404 on /collect	Wrong URL or path	Confirm the exact collector URL with your platform team
Status 401/403	Authentication issue	Contact platform team to verify collector access
Faro not defined error	Import order issue	Ensure faro.js import is the very first line in main.jsx / index.js
No traces in Tempo	TracingInstrumentation missing	Confirm new TracingInstrumentation() is in the instrumentations array